PYTHON · DATA ANALYSIS

Data Manipulation with Pandas

Cleaning, transforming, and analyzing data in Python

A practical introduction to the pandas library

GETTING STARTED

What is Pandas?

An open-source Python library for fast, flexible data manipulation and analysis.

Used everywhere: data cleaning, analysis, and machine learning.

Quick Start

```
import pandas as pd

df = pd.DataFrame(data)
print(df.head())
```

A DataFrame looks like this:

id	name	score
1	Asha	88
2	Ravi	92
3	Meera	79
4	Sam	95

Rows = records

Columns = fields

Index = labels

CORE STRUCTURES

Series vs. DataFrame

Series

A one-dimensional labeled array — a single column of data.

score
88
92
79
95

```
s = pd.Series(
  [88, 92, 79, 95]
)
```

DataFrame

A two-dimensional labeled table, like a spreadsheet.

name	score
Asha	88
Ravi	92
Meera	79

```
df = pd.DataFrame(
  data)
```

LOADING DATA

Reading & Writing Files

Read or write almost any tabular format in one line.

CSV

```
pd.read_csv("data.csv")
```

Excel

```
pd.read_excel("data.xlsx")
```

JSON

```
pd.read_json("data.json")
```

Example

```
df = pd.read_csv("sales.csv")       # load
df.to_csv("clean_sales.csv")        # save
df.to_excel("report.xlsx")          # export
```

EXPLORING DATA

Selecting & Filtering Data

loc & iloc

Select by label or position.

Boolean masks

Filter rows using a condition.

Example

```
df.loc[0, "name"]       # by label
df.iloc[0, 1]           # by position

df[df["score"] > 85]   # filter
```

`Filter: score > 85`

name	score
Asha	88
Ravi	92
Meera	79
Sam	95

name	score
Asha	88
Ravi	92
Sam	95

TRANSFORMING DATA

Adding & Modifying Columns

Create or transform columns without writing loops.

Vectorized math

Operate on whole columns at once.

apply()

Run a custom function on a column.

Example

```
df["bonus"] = df["score"] * 0.1
df["grade"] = df["score"].apply(
    lambda x: "A" if x > 85 else "B")
```

Result

name	score	bonus	grade
Asha	88	8.8	A
Ravi	92	9.2	A
Meera	79	7.9	B
Sam	95	9.5	A

SUMMARIZING DATA

GroupBy & Aggregation

Split data into groups, apply a function, then combine the results.

Example

```
df.groupby("dept")
  ["score"].mean()

df.groupby("dept").agg(
  {"score": "max"})
```

Also: sum(), count(), min(), std()

Grouped by “dept” → mean score

dept	avg_score
Engineering	91.5
Design	83.0
Sales	87.2

Merging & Joining DataFrames

Combine DataFrames on a shared key — just like a SQL JOIN.

id	name
1	Asha
2	Ravi

employees

+

id	dept
1	Eng
2	Sales

departments

→

id	name	dept
1	Asha	Eng
2	Ravi	Sales

merged result

Example

```
pd.merge(employees, departments, on="id")
# how: "inner", "left", "right", "outer"
```

DATA CLEANING

Handling Missing Data

Real-world data has gaps. Pandas makes it easy to find, drop, or fill them.

name	score
Asha	88
Ravi	NaN
Meera	79
Sam	NaN

Missing values appear as NaN

Example

```
df.isna().sum()       # count missing
df.dropna()           # remove NaN rows
df.fillna(0)          # fill with value
df["score"].fillna(
    df["score"].mean())
```

Tip: use fillna() when data is precious, dropna() otherwise.

CONCLUSION

Pandas at a Glance

Series & DataFrame: the foundation of tabular data

Read and write CSV, Excel, or JSON in one line

loc, iloc & masks make selecting data easy

groupby() powers split-apply-combine summaries

merge() combines datasets like a SQL join

fillna() & dropna() keep your data clean